

Obecny stan projektu

ocena	Aplikacja CRUD z wdrożeniem w chmurze z użyciem potoku CI/CD	Uwzględnione aspekty bezpieczeństwa zarówno przy przesyłaniu jak i przechowywaniu danych (https, szyfrowanie haseł)	Wdrożenie w postaci maszyny wirtualnej (użycie Terraform/Ansible) lub kontenera Dockera (docker compose)	Wdrożenie w architekturze mikrousług - min. Dwie komunikujące się ze sobą maszyny wirtualne lub kontenery Dockera	Wdrożenie uzupełnione o testy (jednostkowe, e2e) w ramach potoku CI/CD
	dst				
	dst+				
	db				
	db+				
	bdb				

Poziom dst: Aplikacja CRUD - Osobisty organizator zadań

Funkcjonalności aplikacji i szczegóły implementacyjne

Prosta aplikacja webowa pozwalająca na organizowanie swoich zadań.

Stack technologii:

- Frontend: React (Vite)
- Backend: Node.JS + Express
- Baza danych: SQLite (poziom dst/dst+/db)

Funkcjonalności aplikacji:

- Konta użytkownika: rejestracja konta, logowanie do konta.
- Dashboard: menedżer zadań pozwalający na dodawanie, modyfikowanie, usuwanie zadań

Atrybuty zadania:

- Nazwa
- Krótki opis
- Priorytet (b. wysoki/wysoki/średni/niski)
- Deadline (data)
- Tagi
- Progres: nierozpoczęte/w trakcie/zakończone
- Użytkownicy i zadania są przechowywane w bazie danych SQLite na backendzie.
- Filtrowanie według progresu, priorytetu.
- Sortowanie według deadline'u, priorytetu.
- Prosta wyszukiwarka zadań.

Strony frontendowe:

- / : logowanie (strona domyślna) -> Login.jsx
- /register : zakładanie konta -> Register.jsx

- `/dashboard` : menedżer z zadaniami -> `Dashboard.jsx` `App.jsx` : pobieranie tokena, routowanie

`/dashboard` można wyświetlić tylko po zalogowaniu. Żeby móc wyświetlić stronę domyślną i `/register`, należy nie być zalogowanym. Domyślna strona przekierowuje do `/dashboard` jeśli jest się zalogowanym.

Backend w `server.js` :

- `POST /api/auth/register` : rejestracja konta
- `POST /api/auth/login` : logowanie do konta
- `GET /api/auth/verifytoken` : weryfikacja tokenu tożsamości
- `POST /api/tasks` : dodanie nowego zadania (Create)
- `GET /api/tasks` : pobranie zadań danego użytkownika (Read)
- `PUT /api/tasks/:id` : modyfikacja zadania (Update)
- `DELETE /api/tasks/:id` : usunięcie danego zadania (Delete)

Struktura bazy danych:

Baza danych zdefiniowana w backendzie z użyciem biblioteki `sqlite3`.

Tabela `users` przechowująca użytkowników i tabela `tasks` przechowująca zapisane zadania.

```
db.serialize(() => {
  db.run(`
    CREATE TABLE IF NOT EXISTS users (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      email TEXT UNIQUE,
      password TEXT
    )
  `);

  db.run(`
    CREATE TABLE IF NOT EXISTS tasks (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      userid INTEGER,
      title TEXT,
      description TEXT,
      detailedDescription TEXT,
      priority TEXT,
      deadline TEXT,
      tags TEXT,
      progress TEXT,
      FOREIGN KEY(userid) REFERENCES users(id)
    )
  `);
});
```

Organizator zadań

Wyloguj

Dodaj zadanie

Sortowanie

Szukaj zadania...

Priorytet

☒ bardzo wysoki

☒ wysoki

☒ średni

☒ niski

Progres

☒ Nierozpoczęte

☒ W trakcie

☒ Zakończone

średni

Zrobić zadanie z matematyki

Metody numeryczne

Deadline: 2026-04-07

#studia

nierozpoczęte

Usuń

Edytuj

bardzo wysoki

Test test test

test test

Deadline: brak

nierozpoczęte

Usuń

Edytuj

wysoki

Wynieść śmieci

Deadline: 2026-04-03

nierozpoczęte

Usuń

Edytuj

niski

Testowe zadanie

Testowe zadanie Testowe zadanie

Deadline: brak

w trakcie

Usuń

Edytuj

niski

Zrobić zadanie z analizy
danych

Zadanie 3

Deadline: 2026-10-15

#studia

zakończone

Usuń

Edytuj

Wdrożenie w chmurze z użyciem potoku CI/CD

Zarówno backend, jak i frontend są przechowywane w zdalnym repozytorium:

<https://github.com/sluczakk/projekt-zaliczeniowy-tc>

Wdrożenie aplikacji jako jeden zasób na Azure:

Wykorzystywany zasób to **Azure Web App**.

Ustawienia wdrożenia - zasób Azure jest połączony z wyżej wymienionym repozytorium:

[Settings](#) [Containers \(new\)](#) [Logs](#) [FTPS Credentials](#)

 Save  Discard  Refresh  Browse  Sync  Send us your feedback

Deploy and build code from your preferred source and build provider. [Learn more](#)

Source	GitHub Disconnect
GitHub	
Organization	sluczakk
Repository	projekt-zaliczeniowy-tc
Branch	main
Build	
Build provider	GitHub Actions
Runtime stack	Node
Version	24-lts

Automatyczne wdrożenie dzieje się dzięki **GitHub Actions** - narzędzie wbudowane w GitHub, które pozwala automatyzować różne zadania takie jak budowanie aplikacji, testy i wdrażanie.

Repozytorium w folderze `.github/workflows` zawiera plik `.yaml`, który pozwala na automatyczną integrację i wdrażanie w chmurze. Azure pozwala na automatyczne wygenerowanie szablonu takiego pliku dla danych ustawień zasobu (jak i automatyczne ustawienie np. GitHub Secrets, używanych przez GitHub Actions do logowania się do Azure), jednak czasami potrzebne są zmiany by proces działał dokładnie tak jak chcemy.

```
name: Build and deploy Node.js app to Azure Web App - projekt-zaliczeniowy-tc

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write

    steps:
```

```

- name: Checkout repo
  uses: actions/checkout@v4

- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20.x'

- name: Install frontend dependencies
  working-directory: ./frontend
  run: npm install

- name: Build frontend
  working-directory: ./frontend
  run: npm run build

- name: Install backend dependencies
  working-directory: ./backend
  run: npm install

- name: Copy frontend build into backend/public
  run: |
    rm -rf ./backend/public
    mkdir -p ./backend/public
    cp -r ./frontend/dist/* ./backend/public/

- name: Login to Azure
  uses: azure/login@v2
  with:
    client-id: ${ secrets.AZUREAPPSERVICE_CLIENTID_C7A8EA3EA8444BA48281370D14E606A3 }
    tenant-id: ${ secrets.AZUREAPPSERVICE_TENANTID_04345CBBA8914841BC5776848DFC5E45 }
    subscription-id: ${ secrets.AZUREAPPSERVICE_SUBSCRIPTIONID_D471CE20E0CF419D8AD78A44D1D406F5 }

- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v3
  with:
    app-name: 'projekt-zaliczeniowy-tc'
    slot-name: 'Production'
    package: ./backend

```

Opiszę pokrótce najważniejsze elementy:

```

on:
  push:

```

```
branches:  
  - main
```

Workflow jest uruchamiany automatycznie przy każdym pushu do gałęzi `main`. Oznacza to, że zmiany w kodzie nie wymagają ręcznego budowania i wysyłania plików do Azure; wystarczy zaktualizować repozytorium, a integracja i wdrażanie nastąpią automatycznie.

Kolejne kroki potoku CI/CD:

```
- name: Checkout repo  
  uses: actions/checkout@v4
```

Pobranie aktualnego repozytorium z kodem źródłowym.

```
- name: Set up Node.js  
  uses: actions/setup-node@v4
```

Konfiguracja środowiska Node.js, które jest wymagane do uruchomienia aplikacji oraz wykonania procesu budowania frontendu.

```
- name: Install frontend dependencies  
  working-directory: ./frontend  
  run: npm install  
  
- name: Build frontend  
  working-directory: ./frontend  
  run: npm run build
```

Zainstalowanie wymaganych zależności potrzebnych do uruchomienia frontendu. Frontend jest następnie budowany do wersji produkcyjnej. W wyniku tego kroku powstają statyczne pliki (HTML, CSS, JS) w katalogu `dist`, które mogą być serwowane przez backend.

```
- name: Install backend dependencies  
  working-directory: ./backend  
  run: npm install
```

Instalacja zależności backendu.

```
- name: Copy frontend build into backend/public  
  run: |  
    rm -rf ./backend/public  
    mkdir -p ./backend/public  
    cp -r ./frontend/dist/* ./backend/public/
```

Zbudowane pliki frontendu są kopiowane do katalogu `backend/public`.

Dzięki temu backend (Express) może serwować frontend jako statyczne pliki, co pozwala uruchomić całą aplikację w jednym środowisku Azure Web App.

```
- name: Login to Azure
  uses: azure/login@v2
  with:
    client-id: ${ secrets.AZUREAPPSERVICE_CLIENTID_C7A8EA3EA8444BA48281370D14E606A3 }
    tenant-id: ${ secrets.AZUREAPPSERVICE_TENANTID_04345CBBA8914841BC5776848DFC5E45 }
    subscription-id: ${ secrets.AZUREAPPSERVICE_SUBSCRIPTIONID_D471CE20E0CF419D8AD78A44D1D406F5 }
```

Logowanie do Azure wraz ze wskazaniem zasobu, gdzie mają zostać dostarczone pliki.

```
- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v3
  with:
    app-name: 'projekt-zaliczeniowy-tc'
    slot-name: 'Production'
    package: ./backend
```

Wdrożenie do Azure.

Do platformy wysyłany jest katalog `./backend`, który zawiera:

- kod backendu
- zainstalowane zależności
- zbudowany frontend w katalogu `public`.

Podsumowując, dzięki GitHub Actions po `git push` do repozytorium następuje:

1. pobranie repozytorium
2. budowa frontendu
3. instalacja backendu
4. połączenie frontendu i backendu
5. logowanie do azure
6. wdrożenie aplikacji

Integracja z bazą danych: Baza danych jest obecnie plikiem `sqlite` zarządzanym poprzez backend.

Warto zauważyć, że w wypadku Azure Web App dostępna przestrzeń dyskowa jest ograniczona (ok. 1GB dla darmowego planu). Dla tego projektu jest to wystarczające, jednak na potrzeby produkcyjne większość systemów bardziej rozdziela backend od bazy danych, np. korzystając z Azure SQL, co zapewnia lepszą skalowalność.

Poziom dst+: kwestie bezpieczeństwa

W aplikacji uwzględniłem następujące przykładowe kwestie bezpieczeństwa:

Wymagania co do hasła

Hasło musi mieć minimum 8 liter i zawierać dużą literę, małą literę, cyfrę i znak specjalny.

```
function isPasswordStrong(password) {
  return (
    password.length >= 8 &&
    /[A-Z]/.test(password) && // duża litera
    /[a-z]/.test(password) && // mała litera
    /[0-9]/.test(password) && // cyfra
    /^[^A-Za-z0-9]/.test(password) // znak specjalny
  );
}
```

Szyfrowanie haseł

Hasła użytkowników są przechowywane w postaci zahaszowanej z użyciem algorytmu bcrypt (biblioteka bcryptjs), co uniemożliwia ich odczytanie nawet w przypadku dostępu do bazy danych.

Przy rejestracji: `bcrypt.hashSync(password, 10)` .

Przy logowaniu: `const isMatch = await bcrypt.compare(password, user.password);`

Przykładowe zaszyfrowane hasła:

	<u>id</u>	email	password
	Filtr	Filtr	Filtr
1	1	test@test.com	\$2b\$10\$sOTC0fUmIE11oSQs2Q9p1.xf5Z174zs9eCGBf98UwDIjAfxncv.hC
2	2	test2@test.com	\$2b\$10\$PXvJhNDwb5B4BWfYK8yZQu4sh9pUCrZZHpkPlWG0YoYBpnAp2U3vm

Weryfikacja tożsamości przy modyfikowaniu danych

Kiedy użytkownik się loguje, backend tworzy token i wysyła go na frontend, który go przechowuje. Tokeny są tworzone za pomocą biblioteki `jsonwebtoken` .

```
const token = jwt.sign(
  { id: user.id, email: user.email },
  SECRET_KEY
);
```

Token jest podpisany tajnym kluczem przechowywanym po stronie backendu, dzięki czemu użytkownik nie jest w stanie samodzielnie zmodyfikować jego zawartości.

Przy akcjach modyfikujących bazę danych wymagających weryfikacji tożsamości użytkownika, frontend wysyła token. Token ten jest weryfikowany przez backend:


```
// weryfikacja tokena
function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) return res.status(401).json({ message: "Brak tokenu" });

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, SECRET_KEY);
    req.user = decoded;
    next();
  } catch {
    return res.status(401).json({ message: "Nieprawidłowy token" });
  }
}
```

Jeżeli użytkownik wyśle nieprawidłowy lub zmodyfikowany token, zapytania nie zostaną wykonane.

W tych zapytaniach wykorzystywany jest user id z podpisanego tokena, np.:

```
app.get("/tasks", authMiddleware, (req, res) => {
  db.all(
    `SELECT * FROM tasks WHERE userid = ?`,
    [req.user.id],
```

Dzięki temu użytkownik ma dostęp wyłącznie do swoich danych i nie jest możliwe modyfikowanie danych należących do innych użytkowników, nawet w przypadku ręcznej manipulacji zapytaniami HTTP.

Zapobieganie SQL injection - parametryzacja

Aby zapobiec wstrzykiwaniu kodu SQL, stosowane są zapytania parametryzowane (ze znakiem zapytania ?). Dzięki parametryzacji dane wprowadzane przez użytkownika nie są interpretowane jako kod SQL, lecz jako wartości.

Przykład bezpiecznego zapytania (czytanie userid z tokena poprzez `authMiddleware` wspomniane wcześniej + parametryzacja):

```
app.get("/tasks", authMiddleware, (req, res) => {
  db.all(
    `SELECT * FROM tasks WHERE userid = ?`,
    [req.user.id],
    (err, rows) => {
      if (err) return res.status(500).json({ message: "DB error" });

      res.json({ tasks: rows });
    }
  )
})
```

```
);  
});
```

Przykład niebezpiecznego zapytania:

```
app.get("/tasks-unsafe", (req, res) => {  
  const { userid } = req.query;  
  const query = `SELECT * FROM tasks WHERE userid = ${userid}`;  
  
  db.all(query, [], (err, rows) => {  
    if (err) return res.status(500).json({ message: "DB error" });  
  
    res.json({ tasks: rows });  
  });  
});
```

W tym przypadku pozwalamy na wprowadzenie dowolnego `userid` zamiast czytania go z tokena, więc możliwe jest wprowadzenie dowolnej wartości jako `userid` - pozwala to na SQL injection, co można przetestować poniższym skryptem w Pythonie:

```
import requests  
  
url = "http://localhost:5000/tasks-unsafe"  
  
params = {  
  "userid": "1 OR 1=1"  
}  
  
response = requests.get(url, params=params)  
  
print("Status:", response.status_code)  
print("Response:", response.json())
```

Zwrócenie wszystkich zadań z bazy danych:

```
PS F:\sectest> python test.py  
Status: 200  
Response: {'tasks': [{ 'id': 7, 'userid': 1, 'title': 'xzczx', 'description': 'czxcxzc', 'detailedDescription': 'xzasdas', 'priority': 'medium', 'deadline': '2024-01-01', 'tags': ['hehge'], 'progress': 'done'}, { 'id': 8, 'userid': 1, 'title': 'czxczxc', 'description': 'xzczx', 'detailedDescription': 'zxczx', 'priority': 'very-high', 'deadline': '2020-01-01', 'tags': [], 'progress': 'in-progress'}, { 'id': 9, 'userid': 1, 'title': 'asdasdasd', 'description': 'asdasd', 'detailedDescription': 'asdasdasdas', 'priority': 'low', 'deadline': '', 'tags': ['hhhhh'], 'progress': 'in-progress'}, { 'id': 10, 'userid': 1, 'title': 'adssa', 'description': 'asdas', 'detailedDescription': '', 'priority': 'high', 'deadline': '2026-03-03', 'tags': [], 'progress': 'todo'}, { 'id': 11, 'userid': 3, 'title': 'dasdas', 'description': 'asdas', 'detailedDescription': 'dasdasdasd', 'priority': 'medium', 'deadline': '', 'tags': [], 'progress': 'todo'} ]}
```

Poziom db: wdrożenie w postaci kontenera Dockera

Konfiguracja kontenera

Na tym poziomie dochodzą dwa nowe pliki - `Dockerfile` i `docker-compose.yml`.

Dockerfile :

```
# Budowa frontendu
FROM node:20-bookworm AS frontend-build
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ .
RUN npm run build

# Budowa backendu
FROM node:20-bookworm
WORKDIR /app

# narzędzia potrzebne do kompilacji natywnych modułów (sqlite3)
RUN apt-get update && apt-get install -y python3 make g++ && rm -rf
/var/lib/apt/lists/*

COPY backend/package*.json ./

# natywna kompilacja modułu
RUN npm ci --omit=dev --build-from-source

COPY backend/ .
COPY --from=frontend-build /app/dist ./public

RUN mkdir -p /app/database

ENV NODE_ENV=production
ENV DATABASE_PATH=/home/data/database.sqlite

EXPOSE 3000

CMD ["node", "server.js"]
```

Krótkie wytłumaczenia:

FRONTEND

```
1 FROM node:20-bookworm AS frontend-build
2 WORKDIR /app
3 COPY frontend/package*.json ./
4 RUN npm ci
5 COPY frontend/ .
6 RUN npm run build
```

- 1 Używamy obrazu Node.js, wersja 20 Debian bookworm.
- 2 Ustawiamy katalog roboczy w kontenerze.
- 3 Kopiujemy package.json i package-lock.json z frontendu
- 4 Instalujemy zależności (z wyżej wymienionych plików)

5 Kopiujemy frontend.

6 Budujemy frontend (zamieniamy React na statyczne strony).

BACKEND

```
1 FROM node:20-bookworm
2 WORKDIR /app

# narzędzia potrzebne do kompilacji natywnych modułów (sqlite3)
3 RUN apt-get update && apt-get install -y python3 make g++ && rm -rf
/var/lib/apt/lists/*

4 COPY backend/package*.json ./

# natywna kompilacja modułu
5 RUN npm ci --omit=dev --build-from-source

6 COPY backend/ .
7 COPY --from=frontend-build /app/dist ./public

8 RUN mkdir -p /home/data/

9 EXPOSE 3000

10 ENV NODE_ENV=production
11 ENV DATABASE_PATH=/home/data/database.sqlite

12 CMD ["node", "server.js"]
```

1 Używamy obrazu Node.js, wersja 20 Debian bookworm.

2 Ustawiamy katalog roboczy w kontenerze.

3 Pobieramy narzędzia potrzebne do kompilacji natywnej modułów: `python3`, `make` i `g++`. Bez tego kroku `npm ci` nie zadziała, gdyż Docker nie znajdzie biblioteki `glibc` pasującej do wykorzystanej wersji modułu `sqlite`.

```
backend-1 | Error: /lib/x86_64-linux-gnu/libm.so.6: version `GLIBC_2.38' not found
(required by /app/node_modules/sqlite3/build/Release/node_sqlite3.node)
```

4 Kopiujemy `package.json` i `package-lock.json` z backendu

5 Instalujemy zależności. Flaga `--build-from-source` oznacza, że jeśli jakaś paczka ma natywny kod (C/C++) - w tym przypadku `sqlite3`, to Docker zamiast użyć gotowego binarnego pliku skompiluje ją lokalnie.

6 Kopiujemy backend

7 Kopiujemy zbudowany frontend do backendu, by móc go serwować poprzez backendowy server

8 Tworzymy katalog na bazę danych SQLite

9 Ustawiamy port używany przez kontener na 3000

10, 11 Zmienne środowiskowe.

12 Po starcie kontenera uruchamiamy serwer poprzez komendę `node server.js`

`docker-compose.yml` (jeśli korzystamy z `docker compose`):

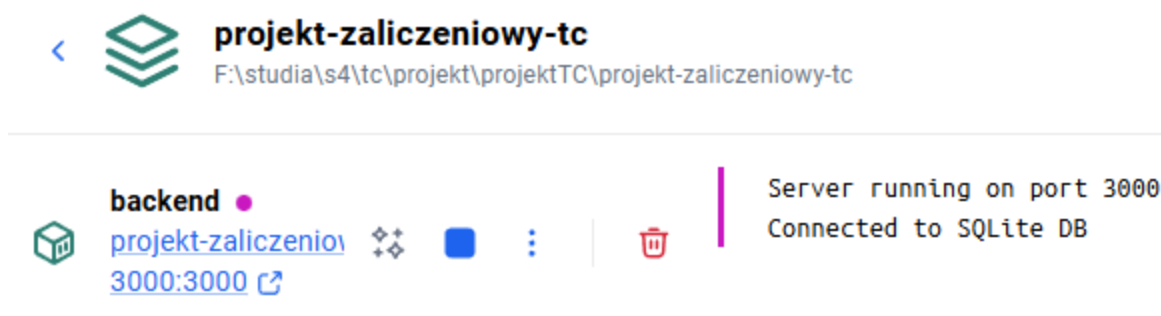
```
services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    volumes:
      - sqlite-data:/home/data

volumes:
  sqlite-data:
```

Ustawiamy usługi. Jako iż frontend jest budowany i serwowany poprzez backend, jedyną usługą jest backend. Docker buduje obraz z wyżej opisanego Dockerfile i konfiguruje porty. W `volumes` informujemy Dockera, że korzystamy z wolumenu, czyli miejsca na trwałe dane - w tym przypadku baza danych - i przypisujemy do folderu w kontenerze; nie chcemy, żeby plik z bazą danych zniknął po restarcie kontenera.

Po uruchomieniu serwisu, `volumes` tworzy wolumen `sqlite-data`, jeśli nie został jeszcze stworzony.

Jak mamy już oba pliki, wystarczy uruchomić silnik Dockera i wywołać w katalogu komendę `docker compose up --build`. Wynikiem będzie uruchomienie skonfigurowanego kontenera z aplikacją.



Wdrożenie w Azure Web App

Żeby automatycznie budować kontener za każdym `git push` do repozytorium, ponownie skorzystamy z GitHub Actions i Azure Web App.

Mechanizm działa podobnie jak poprzednio, z tą różnicą że Azure Web App dostaje kontener, a nie same pliki:



Save



Discard



Refresh



Browse



Sync



Send us your feedback



Troubleshoot

Source

GitHub

[Disconnect](#)**GitHub Actions**

Organization

sluczakk

Repository

projekt-zaliczeniowy-tc

Branch

[main](#)**Registry settings**

Server URL

projekttcacr.azurecr.io

Login

projekttcacr

Image

tccruddapp

Tag

Tagged with GitHub commit ID

W repozytorium na GitHubie dodałem nowy plik `yaml`, który podobnie jak wcześniejszy pozwala na automatyczne wdrażanie w ramach potoku CI/CD. Różnica jest taka, że zamiast budować frontend i backend, budowany jest bezpośrednio image Dockera z `Dockerfile` w repozytorium i wysyłany do wcześniej ustawionego zasobu Azure Container Registry.

```
jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read #This is required for actions/checkout

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Log in to container registry
        uses: docker/login-action@v2
        with:
```

```

    registry: projekttcacr.azurecr.io/
    username: ${
secrets.AZUREAPPSERVICE_CONTAINERUSERNAME_38BF90053ED74B51947F05E1E44357F3 }}
    password: ${
secrets.AZUREAPPSERVICE_CONTAINERPASSWORD_E6D7FBB655324FC591A6651B92361755 }}

- name: Build and push container image to registry
  uses: docker/build-push-action@v3
  with:
    context: .
    push: true
    tags: projekttcacr.azurecr.io/${
secrets.AZUREAPPSERVICE_CONTAINERUSERNAME_38BF90053ED74B51947F05E1E44357F3
}}/tccruddapp:${{ github.sha }}
    file: ./Dockerfile

```

Po zbudowaniu image'u kontenera, aplikacja jest wdrażana z jego wykorzystaniem.

```

deploy:
  runs-on: ubuntu-latest
  permissions:
    id-token: write #This is required for requesting the JWT
    contents: read #This is required for actions/checkout

  needs: build

  steps:

    - name: Login to Azure
      uses: azure/login@v2
      with:
        client-id: ${
secrets.AZUREAPPSERVICE_CLIENTID_59737136FBF146C395A77474FF5F3A15 }}
        tenant-id: ${
secrets.AZUREAPPSERVICE_TENANTID_2D08C5D9E13B4A498B38041A5EA4A739 }}
        subscription-id: ${
secrets.AZUREAPPSERVICE_SUBSCRIPTIONID_AA69E80A2992472DB5E7D7E8FD7B142E }}

    - name: Deploy to Azure Web App
      id: deploy-to-webapp
      uses: azure/webapps-deploy@v2
      with:
        app-name: 'projekt-zaliczeniowy-tc-kontener'
        slot-name: 'Production'
        images: 'projekttcacr.azurecr.io/${
secrets.AZUREAPPSERVICE_CONTAINERUSERNAME_38BF90053ED74B51947F05E1E44357F3
}}/tccruddapp:${{ github.sha }}'

```

Jest jednak kilka różnic pomiędzy korzystaniem z kontenera lokalnie, a poprzez Azure Web App. Azure Web App:

- ignoruje `docker-compose.yml`
- nie korzysta z wolumenów Dockera
- po restarcie kontenera zachowuje się tylko folder `/home`

Jest to powód dla którego w `Dockerfile` ustawiamy katalog bazy danych na `/home/data`. Jest to wystarczające na mały projekt, jednak tak jak wspomniałem wcześniej, w poważniejszych systemach stosuje się inne rozwiązania.

W przypadku wersji z kontenerem, w ustawieniach zasobu należy włączyć `WEBSITES_ENABLE_APP_SERVICE_STORAGE`, by baza danych była trwała (domyślnie jest ustawione na `false`).

App settings

Connection strings

🔍 Search

+

 Add

↺

 Refresh

👁

 Show values

✎

 Advanced edit

↩

 Pull reference values

Name	Value
DOCKER_REGISTRY_SERVER_PASSWORD	👁 Show value
DOCKER_REGISTRY_SERVER_URL	👁 Show value
DOCKER_REGISTRY_SERVER_USERNAME	👁 Show value
WEBSITES_ENABLE_APP_SERVICE_STORAGE	👁 false

..

Poziom db+ - wdrożenie w architekturze mikrouslug - dwa kontenery Dockera

Wdrożenie jako dwa kontenery

W poprzedniej wersji aplikacja była wdrazana w postaci jednego kontenera - część frontendowa była budowana i serwowana poprzez backend. Na tym poziomie wykorzystam dwa kontenery i ustawie komunikację miedzy dwoma zasobami Azure.

W tym wypadku skorzystam z Azure Container App zamiast Azure Web App.

Utworzyłem dwie instancje Azure Container App.

Search

Refresh Send us your feedback

- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Resource visualizer
- Application
 - Revisions and replicas
 - Containers
 - Scale
 - Volumes
- Settings
 - Deployment**
 - Development stack
 - Dapr
 - Service Connector (preview)
 - Resiliency (preview)
 - Locks
- Networking
 - Ingress
 - Custom domains
 - CORS
- Security

Continuous deployment Artifact (preview)

Set up GitHub Actions to automatically build and deploy your code to your Container App. Note that every deployment will create a new revision.

GitHub settings

If you can't find an organization or repository, you may need to enable additional permissions on GitHub. [Get more info](#)

Signed in as *	sluczakk
	<button>Change account</button>
Organization *	Select organization
Repository *	Select repository
Branch *	Select branch

Registry settings

With every deployment, you'll get a new image tag that will be stored in the registry of your choice.

Repository source	☒ Azure Container Registry
	☐ Docker Hub or other registries
Subscription *	Azure for Students
Registry *	projekttcacr
Image	tc-dbplus-frontend
Image tag	Tagged with GitHub commit ID (SHA)
OS type	Linux
Dockerfile location ⓘ	Ex: <code>../Dockerfile</code>

Frontend i backend korzystają ze wspólnego środowiska Container Apps Environment. Wspólne środowisko umożliwia komunikację między kontenerami.

Dodatkowa przykładowa kwestia bezpieczeństwa - backend pozwala na komunikację tylko z aplikacjami w tym samym środowisku, dzięki czemu tylko frontend ma do niego dostęp. Ustawienia Ingress:

Ingress ⓘ ☒

Ingress traffic ☒ Limited to Container Apps Environment
Select this option if you want to restrict traffic to this container app from within the Container App Environment

☐ Accepting traffic from anywhere
Select this option if you want to allow traffic to this container app from anywhere

Ingress type ⓘ ☒ HTTP

☐ TCP

Client certificate mode ⓘ ☒ Ignore

☐ Accept

☐ Require

Transport

Insecure connections ☐

Target port ⓘ

Endpoint(s) <https://tc-dbplus-backend.internal.gentlewave-6e79baa5.polandcentral.azurecontainerapps.io>

Session affinity ⓘ ☐

Następnie utworzyłem plik GitHub Actions, który pozwala na automatyczne wdrażanie i integrację podobnie jak na poprzednich poziomach. Z każdym commitem do repozytorium na GitHubie:

- Budowane są obrazy frontendu i backendu i przesyłane do Azure Container Registry
- Utworzone zasoby Azure Container App są aktualizowane by korzystały z najnowszych obrazów z Azure Container Registry

```
name: Build and deploy frontend/backend containers
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
  workflow_dispatch:
```

```
env:
```

```
  ACR: projekttcacr.azurecr.io
```

```
  FRONTEND_IMAGE: tccrud-frontend
```

```
  BACKEND_IMAGE: tccrud-backend
```

```
  RESOURCE_GROUP: resourcegroup1
```

```
  FRONTEND_CONTAINER_APP: tc-dbplus-frontend
```

```
  BACKEND_CONTAINER_APP: tc-dbplus-backend
```

```

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write #This is required for requesting the JWT
      contents: read #This is required for actions/checkout

    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to container registry
        uses: docker/login-action@v3
        with:
          registry: ${ env.ACR }
          username: ${
secrets.AZUREAPPSERVICE_CONTAINERUSERNAME_38BF90053ED74B51947F05E1E44357F3 }}
          password: ${
secrets.AZUREAPPSERVICE_CONTAINERPASSWORD_E6D7FBB655324FC591A6651B92361755 }}

      - name: Build and push frontend image
        uses: docker/build-push-action@v6
        with:
          context: .
          file: ./Dockerfile.frontend
          push: true
          platforms: linux/amd64
          provenance: false
          sbom: false
          build-args: |
            VITE_API_URL=/api
          tags: |
            ${ env.ACR }/${ env.FRONTEND_IMAGE }:latest
            ${ env.ACR }/${ env.FRONTEND_IMAGE }:${ env.github.sha }}

      - name: Build and push backend image
        uses: docker/build-push-action@v6
        with:
          context: .
          file: ./Dockerfile.backend
          push: true
          platforms: linux/amd64
          provenance: false
          sbom: false
          tags: |
            ${ env.ACR }/${ env.BACKEND_IMAGE }:latest
            ${ env.ACR }/${ env.BACKEND_IMAGE }:${ env.github.sha }}

```

```

- name: Azure Login frontend
  uses: azure/login@v2
  with:
    client-id: ${ secrets.TCDBPLUSFRONTEND_AZURE_CLIENT_ID }
    tenant-id: ${ secrets.TCDBPLUSFRONTEND_AZURE_TENANT_ID }
    subscription-id: ${ secrets.TCDBPLUSFRONTEND_AZURE_SUBSCRIPTION_ID }

- name: Deploy frontend Container App
  uses: azure/container-apps-deploy-action@v2
  with:
    containerAppName: ${ env.FRONTEND_CONTAINER_APP }
    resourceGroup: ${ env.RESOURCE_GROUP }
    imageToDeploy: ${ env.ACR }/${ env.FRONTEND_IMAGE }:${ env.github.sha }

- name: Azure Login backend
  uses: azure/login@v2
  with:
    client-id: ${ secrets.TCDBPLUSBACKEND_AZURE_CLIENT_ID }
    tenant-id: ${ secrets.TCDBPLUSBACKEND_AZURE_TENANT_ID }
    subscription-id: ${ secrets.TCDBPLUSBACKEND_AZURE_SUBSCRIPTION_ID }

- name: Deploy backend Container App
  uses: azure/container-apps-deploy-action@v2
  with:
    containerAppName: ${ env.BACKEND_CONTAINER_APP }
    resourceGroup: ${ env.RESOURCE_GROUP }
    imageToDeploy: ${ env.ACR }/${ env.BACKEND_IMAGE }:${ env.github.sha }

```

Do połączenia frontendu z backendem kluczowe jest podanie prawidłowego adresu backendu.

Zmodyfikowałem frontend, by teraz był serwowany przez Nginx (na poprzednich poziomach był serwowany poprzez backend). Adres API wykorzystywany w wersji produkcyjnej (hostowanej na Azure) jest teraz zapisany w konfiguracji nginx `nginx.conf`:

```

server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # API proxy do backendu
    location /api/ {
        proxy_pass https://tc-dbplus-backend.internal.gentlewave-
6e79baa5.polandcentral.azurecontainerapps.io/;

        proxy_ssl_server_name on;
    }
}

```

```
    proxy_http_version 1.1;
    proxy_set_header Host tc-dbplus-backend.internal.gentlewave-
6e79baa5.polandcentral.azurecontainerapps.io;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location / {
    try_files $uri $uri/ /index.html;
}
}
```

(Adres " /api/ " ustawiony na produkcji w pliki GitHub Actions przekierowuje do adresu backendu na Azure)